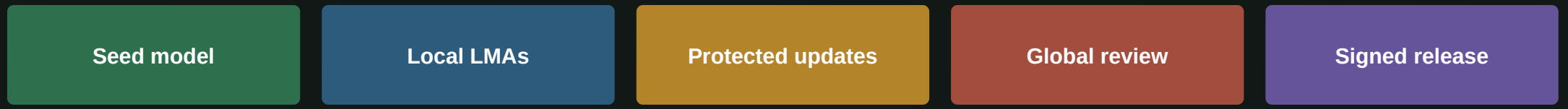
FEDERATED HEALTHCARE USE CASE

Dagents for Federated Hospital Learning

Send a signed seed model to hospitals, train and evaluate locally, aggregate protected updates, and release only after cross-site review.



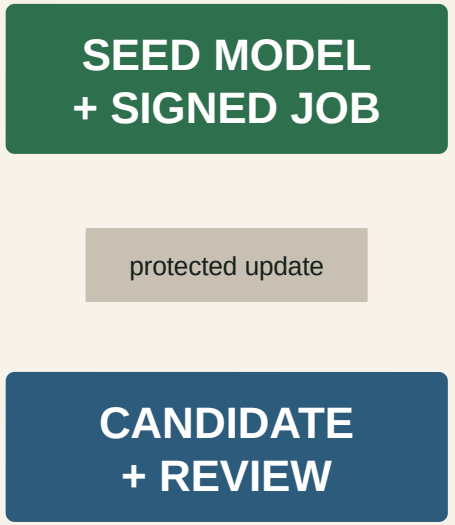
Pradyun Devarakonda
Professor presentation

THE PROPOSAL

One control plane; hospital data stays local.

Dagents coordinates a five-hospital, sepsis-specific research pilot. A specialist federated engine performs protected aggregation.

- First case: improve and evaluate one sepsis warning model across five hospitals.
- GMA: selects sites, sends signed jobs, records rounds, and coordinates release or rollback.
- LMA: validates and runs beside local records; patient rows and detailed evidence remain local.
- Specialist layer: NVIDIA FLARE or an equivalent engine handles federated aggregation and privacy mechanisms.



THE PROBLEM

Pooling raw hospital records is not the only way to learn.

The system must handle data boundaries, site differences, update privacy, and safe operations—not just model accuracy.

Data cannot simply move

Privacy, contracts, sovereignty, and breach risk make a central patient-data copy a poor default.

Hospitals do not match

Populations, units, systems, labels, and workflows differ. One global setting may not fit every site.

Updates can still leak

Weights and gradients are not automatically anonymous. Aggregation and privacy controls still matter.

Operations decide safety

Identity, code approval, retries, versions, evidence, canaries, and rollback determine whether learning is controllable.

USE-CASE BOUNDARY

Start narrow enough to prove the whole system.

Illustrative research pilot: five hospitals, adult inpatient wards, sepsis first, past-data testing followed by silent live evaluation.

One clinical question

Near-term sepsis deterioration risk. A clinical committee defines the reference event, horizon, exclusions, and safe use.

Approved seed

Start from public, de-identified, consented, synthetic, or otherwise permitted data—not a central copy of hospital records.

No automatic care

Past-data and silent live stages first. No diagnosis, treatment, or clinician-facing warning without a separate approved study.

One condition

Later cases could cover stroke, heart attack, sudden kidney injury, or breathing decline. Each needs its own model, workflow, owner, and safety evidence.

WALKTHROUGH

What happens to one patient's record.

The record stays in the hospital. Only a number leaves.

ARRIVES

A patient is admitted. Vital signs, laboratory results, and notes land in the hospital's own systems, exactly as they do today.

STAYS LOCAL

The LMA builds the approved features inside the hospital. Nothing is copied out, and the hospital can refuse the job.

BECOMES MATH

The local model trains or scores. This patient turns into part of a total, not a row anyone can read.

LEAVES SMALL

Only the protected update and the agreed counts leave. Nobody outside can rebuild the patient from them.

The record never moves. The learning does.

DATA BOUNDARY

Exactly what crosses the hospital boundary.

If you can name every field that leaves, you can govern it. This is the whole list.

LEAVES THE HOSPITAL

- The protected model update
- Counts and scores that were agreed in advance
- Round number, model version, and status
- A pointer to evidence that stays at the hospital

NEVER LEAVES

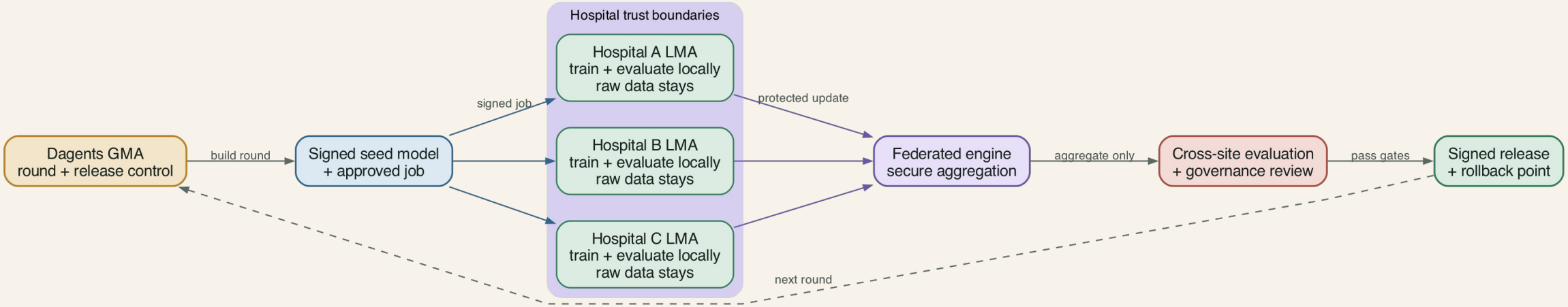
- Patient rows
- Names, identifiers, and dates of birth
- Results for any single patient
- Notes and images
- Anything not written into the contract

“We only send weights” is a claim. A named list plus a check that blocks everything else is a control.

FEDERATED ARCHITECTURE

Dagents controls the round; a federated engine aggregates.

Signed jobs go out, protected updates come back, and raw records plus patient-level traces stay at each hospital.



ONE FEDERATED ROUND

A round moves code and updates—not patient records.

The coordinator never treats ‘weights only’ as a privacy guarantee.

DISTRIBUTE

GMA selects approved sites and sends a signed seed, code hash, data contract, privacy profile, and round ID.

TRAIN LOCALLY

Each LMA checks inputs, trains or evaluates inside the hospital, and bounds its contribution.

PROTECT + AGGREGATE

Secure aggregation and a minimum-participant rule reveal only the allowed combined update.

EVALUATE + DECIDE

The candidate returns to sites; cross-site evidence and governance decide release or rejection.

Any failed gate cancels the round; the current approved model keeps running.

RELEASE LIFECYCLE

Treat every model version as a recoverable release.

Federated training creates candidates; governance creates releases.

BASELINE

Record the seed model, code, features, dependencies, intended use, approved sites, and safe fallback.

INCREMENT

Create a round manifest; collect bounded updates and approved aggregate results.

VERIFY

Re-evaluate at every site; check privacy, performance, subgroup, integrity, and operational gates.

RELEASE OR RESTORE

Sign a candidate for a small rollout—or keep and restore the last known-good model and settings.

No candidate silently replaces the approved model.

LOCAL AGENT (LMA)

The LMA is the hospital’s enforcement point.

It accepts only approved jobs, touches only approved fields, and keeps the detailed trace under hospital control.

VERIFY JOB

Check signer, code hash, dependencies, condition, allowed fields, privacy settings, and local approval. The hospital may reject.

PREPARE LOCALLY

Build the approved feature view; validate units, time windows, missingness, cohort, and label availability.

TRAIN + EVALUATE

Run an isolated workload, calculate local measures, and clip or bound the site contribution.

PUBLISH MINIMUM

Send only the protected update, allowed measures, status, versions, and a local evidence pointer.

Detailed records, features, and patient-level predictions never belong in the GMA result.

GLOBAL AGENT (GMA)

The GMA controls participation, rounds, and releases.

It coordinates the federation; it does not become a raw patient database.

REGISTER

Track each hospital, LMA, approved use, capability, schema version, certificate, and sharing rule.

PLAN ROUND

Choose participants, minimum threshold, seed version, privacy profile, evaluation plan, and stop conditions.

COORDINATE

Send signed jobs; track acknowledgements, timeouts, dropouts, retries, cancellation, and aggregate status.

RELEASE + ROLLBACK

Collect cross-site evidence, sign a canary release, watch health, or restore the known-good version.

When secure aggregation is required, the GMA should not read any hospital’s individual update.

METHOD BOUNDARIES

Four ‘federated’ ideas solve different problems.

Use them together, but do not treat the names as interchangeable.

Method	What it does	Dagents role / important limit
Federated learning	Trains a shared model from updates computed at each hospital	Use a specialist FL engine; Dagents coordinates jobs, evidence, releases, and rollback
Federated analytics	Computes local counts, statistics, and evaluation; reveals approved aggregates	Best first phase; small groups and repeated queries can still reveal sensitive facts
Federated database	Offers one query layer across remote sources and can push work to the source	Useful inside a hospital; unrestricted cross-site patient queries are out of scope
Central warehouse	Copies data into one place for central processing and analytics	Useful for some needs, but not a required dependency for this time-sensitive path
Backup patterns	Protect versions with baselines, increments, catalogs, checks, and recovery	Operations model only; backup software does not train or validate a clinical model

NVIDIA FLARE EXAMPLE

EXAM shows cross-hospital federation can work.

Evidence for the operating model—not a performance promise for sepsis or Dagents.

20 institutions

The study coordinated hospitals and research sites across several countries.

16,148 cases

Chest X-rays and electronic-record features were used in the research cohort.

No dataset exchange

Sites trained through a client/server federation instead of sending raw site datasets to one repository.

Reported improvement

For the 24-hour task: 16% average AUC improvement and 38% average generalizability improvement versus local models.

The paper reported average AUC above 0.92 and stated the model was research-only and not regulator approved.

WHAT DAGENTS SHOULD LEARN

The lesson is the operating model, not the brand.

A real healthcare federation needs repeatable experiments, site workers, evaluation, privacy controls, and release governance.

Many traceable rounds

The EXAM study ran many experiments. Dagents needs durable round state, inputs, outputs, versions, and reasons.

Local site capability

Each hospital needs approved compute, data preparation, training, evaluation, and evidence storage.

Heterogeneity stays

Shared features do not erase site differences. Every candidate needs site-level and group-level evaluation.

Code + privacy approval

Hospitals should verify job code and data access; secure aggregation and privacy controls remain separate work.

ENTERPRISE BACKUP LESSONS

Backup patterns make model releases safer.

Borrow central policy, local workers, integrity checks, and recovery—not the idea that an update is automatically trustworthy.

Baseline + increments

A full backup plus changed blocks maps to a signed seed package plus bounded round updates.

Catalog + lineage

A backup catalog maps to model, code, feature-contract, round, participant, candidate, and release history.

Integrity checks

Checksums, signatures, reproducible evaluation, and locked releases detect corruption or silent change.

Restore drills

A rollback plan is incomplete until teams regularly prove that the known-good model can be restored.

DAGENTS MAPPING

Dagents supplies the control plane around specialist federation.

Five framework capabilities compose the use case; no single service is the whole solution.

LMA near the data

Verify the job, build local features, train or evaluate, keep detailed evidence, and publish the minimum result.

Pipeline plans

Define repeatable readiness, training, evaluation, packaging, approval, rollout, and recovery steps.

Federated engine

Run FedAvg or another approved algorithm, secure aggregation, privacy mechanisms, and dropout handling.

Model registry

Store signed seeds, candidates, releases, code digests, feature contracts, evidence pointers, and rollback targets.

GMA + core

Register sites, plan rounds, compile workloads, coordinate release, monitor health, pause, and restore.

Controls across every step: identity, least privilege, minimum messages, signatures, retention, evidence, and human approval.

COMPLIANCE + SAFETY

Less data movement does not equal compliance.

Technical controls help, but hospitals and the consortium still own policy, operations, clinical evidence, and accountable decisions.

HIPAA

Keeping records local can reduce copies. Teams still need approved purpose, minimum-necessary decisions, access control, encryption, BAAs, retention, audit, risk analysis, and incident response.

SOC 2

Dagents can produce health, change, and recovery evidence. The organization still needs control owners, access reviews, change management, availability targets, vendor risk work, and an independent examination.

CLINICAL AI SAFETY

Each sepsis model needs a defined intended use, site and group validation, usability testing, clinician authority, monitoring, rollback, and regulatory analysis before care use.

Federated architecture is a control choice—not legal, audit, privacy, or medical approval / 15

ROUND SAFETY GATES

Four gates must pass before a round changes the model.

Any failure cancels the candidate, records the reason, and keeps the current release in service.

1. SCOPE + SITE

Is the hospital, condition, patient group, seed version, feature contract, and purpose approved?

2. CODE + DATA

Did the hospital verify the signer and code hash? Are units, cohort, labels, freshness, and quality acceptable?

3. PRIVACY

Are contributions bounded, participant threshold met, egress allowed, and aggregation/privacy settings correct?

4. RELEASE

Did every site evaluation, subgroup, security, clinical, canary, and rollback check pass?

A successful aggregation is only a candidate—not permission to deploy.

A published way to decide who may see what.

GRAILS keeps the ethics rules out of the normal application code, so they can be changed without rewriting the app.

Two parts

The Rails work out what protection is needed. The Guard applies it. Deciding and enforcing stay separate.

Three questions

How sensitive is the data? How far do we trust who is asking? How much are they asking for?

Added, not rebuilt

It is designed to plug into software that already exists. You do not redesign the app to add ethics to it.

Wider than privacy

Privacy is the paper's worked example. The same shape extends to fairness, transparency, and security.

Access rules based on roles alone cannot express “this person, this field, this purpose, today”. GRAILS can.

Three dials decide the answer.

The same request can get a different answer, depending on where the three dials sit.

HOW SENSITIVE

Low, medium, or high. Set from the data itself and the rules that cover it, such as HIPAA. A ward name and a diagnosis are not the same.

HOW TRUSTED

The paper calls this a Know Your User score, like a bank checking a customer. Built from verified identity, employer, stated purpose, and past record.

HOW MUCH

One value, one patient, one field across everyone, or the whole table. Asking for more should not be as easy as asking for less.

High sensitivity and low trust means heavy protection. Low sensitivity and high trust means show it as it is.

The dials point to a specific action.

A shortened view of the paper's strategy table, written in plain terms.

How sensitive the field is	Asked by low trust	Asked by medium trust	Asked by high trust
High	Hide the value, or replace it with noise	Round it off, or report it as a group	Show it with a little noise added
Medium	Report it as a group only	Show a range, hide the rest	Show it as it is
Low	Add a little noise	Show it as it is	Show it as it is

The point is not the exact technique. The point is that the answer is looked up from a table, not decided inside application code.

GRAILS splits the same way Dagents already splits.

The Rails are planning. The Guard is runtime. That line is the one Dagents is built on.

Rails = planning

Work out the protection plan. Pure, typed, repeatable, and no input or output. This belongs in the OCaml layer, beside the dataset and model planners.

Guard = runtime

Apply the plan to real data and record what was decided. This belongs in the LMA and GMA, where the side effects already are.

GRAILS part	Where it goes in Dagents	Why there
Restriction Planner	OCaml planner module	The paper compares it to a database query planner. Dagents already has that layer.
Ethical Guard	LMA and GMA request path	Enforcement needs real data, real users, and an audit log.
Rules and regulations	Configuration, not code	Policy changes far more often than the framework does.

Four places the guard has to stand.

A check at the edge only is a warning label. A check at every boundary is a control.

BEFORE READING

The LMA checks the job before it touches a single hospital field. An unapproved job stops here.

BEFORE TRAINING

Check the group of patients is large enough and that every field used is on the approved list.

BEFORE SENDING

The update and the counts are checked before anything leaves the hospital network.

BEFORE RELEASE

The GMA checks the candidate model and its evidence before it signs anything as released.

Every check writes down what it decided, why, and which round it belonged to.

GRAILS protects rows. Federation sends updates.

This is the one place the framework has to be extended for this use case, and it is the contribution this project can make.

What it covers today

One value, one row, one column, or a whole table. All of them are data you can point at and read.

What federation adds

A model update is not a row, but it still carries patient signal. It needs to become a fifth thing the rules cover.

Sensitivity becomes

How much patient signal an update can carry out of the hospital.

Trust becomes

The receiving site and the coordinator, not only a named person.

Action becomes

Limit the contribution, require enough sites, add noise, or refuse to send.

This is a real extension, not a restatement of the paper.

One request, from start to finish.

A researcher at one hospital asks for stroke cases so they can check a model.

WHO IS ASKING

A verified staff account, a named study, an approved purpose, and a clean record. Trust comes out as medium.

WHAT THEY WANT

Arrival time, age, and outcome for stroke patients. That is one field across many patients, not one patient.

WHAT THE RULES SAY

Outcome is a high sensitivity field. Medium trust, asked for across a whole column, so group it and hide part of it.

WHAT THEY GET

Age bands instead of ages, grouped outcomes, and a written reason. Enough to check the model, not enough to identify anyone.

Nobody wrote code for this specific request. The plan came from the three dials.

CHOICE

Why Dagents over a framework or SaaS alone?

Best fit: Dagents supplies reusable control and release governance around a specialist federated-learning engine.

Option	Advantages	Tradeoffs
Dagents + FL engine	Source-local LMA boundary; reusable GMA orchestration; portable contracts; evidence and rollback in one design	More integration and operational ownership; privacy, identity, and clinical controls still need production work
FL framework alone	Mature algorithms, simulation, aggregation, and privacy extensions	Needs a surrounding control plane for orchestration, evidence, releases, and product integration
Managed FL SaaS	Fast onboarding, hosted services, and vendor support	Lock-in, recurring cost, deployment constraints, and less control over evidence or recovery
Central warehouse + ML	Familiar central analytics and model tools	Large and delayed data flows, duplicated sensitive records, and a larger governance boundary
One-off custom build	Can match one hospital network exactly	Slow to reuse and maintain; likely to create incompatible contracts for the next condition

Dagents wins on control, portability, and recoverability—not on lowest initial effort / 26

PILOT ROADMAP

Prove federation in stages.

Start with federated analytics; enable training only after the data, privacy, and recovery paths work.

Phase	What happens	Proof needed
1. Define	Approve the sepsis research question, patient group, outcome, horizon, sources, site owners, and stop conditions	Signed clinical and data contract; named owners; no automatic care action
2. Federated analytics	Compute local counts, missingness, units, labels, and seed-model measures; reveal approved aggregates only	Same meaning at every site; minimum-message path; privacy threshold and audit trail
3. Simulate	Exercise complete rounds with approved test data, including site rejection, dropout, retry, bad update, and rollback	Deterministic recovery, signatures, round state, and no individual-update exposure
4. Retrospective train	Train on historical local data; evaluate every candidate locally and compare with local-only and current baselines	Cross-site and subgroup evidence; privacy report; candidate rejected when any gate fails
5. Silent live	Run the approved model on current data without displaying warnings to clinicians	Reliability, drift, alert volume, security, recovery, and separate clinical approval for any next stage

SUCCESS MEASURES

Success means quality, privacy, and recovery.

No single accuracy number proves that the federated system is safe or useful.

Cross-site model evidence

Calibration and discrimination at every hospital; approved group results; comparison with local-only and current baselines.

Minimal data movement

Zero raw patient rows in the GMA path; measured message size; blocked fields; cohort and privacy-threshold compliance.

Reliable rounds

Completion, timeout, dropout, retry, cancellation, signature verification, and time to stop a bad round.

Proof of recovery

Complete seed-to-release lineage; immutable approved versions; successful scheduled rollback and evidence-recovery drills.

RECOMMENDATION

Start with federated analytics; earn the right to train.

Use Dagents as the governed orchestration and release layer around a specialist federated-learning engine.

- Define one sepsis research use, feature contract, participant policy, site owner, and safe fallback.
- Run data-readiness and seed-model evaluation locally; share only protected aggregate measures.
- Integrate NVIDIA FLARE or an equivalent engine behind Dagents round and result contracts.
- Require local code approval, secure aggregation where needed, signed releases, and complete lineage.
- Practice cancellation and rollback before retrospective training—and again before any clinician-facing study.

GLOSSARY

Federated-system terms in plain language.

The terms describe different jobs; none is automatic proof of privacy, security, safety, or compliance.

Term	Plain meaning	Why it matters here
Federated learning	Hospitals train locally and send protected model updates instead of records	Improves one shared model; requires a specialist engine and privacy controls
Federated analytics	Hospitals calculate approved measures locally and reveal combined results	The safest first cross-hospital workload for readiness and evaluation
Federated database	One query layer can reach several remote data sources	Useful for local feature views; can still expose or move detailed results
Secure aggregation	The coordinator learns a combined update without reading each site’s update	Reduces exposure; does not solve poisoning, memorization, or governance alone
Restore point / rollback	A locked known-good release / return to that release after a problem	Backup-inspired control that makes model changes reversible